

전자전기컴퓨터설계실험II

실험 전 보고서

LAB5

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 10월 15일
대상	인터럽트와 PWM
검증 환경	VS Code / NUCLEO-G474RE

1 실험 목적과 관련 이론

실험 목적

NUCLEO-G474RE에서 버튼 입력을 인터럽트로 처리하고 타이머로 LED 점멸과 PWM 출력을 구현한다. 과제 3은 이전 LED 간격 변경 동작을 ISR 방식으로 재현하고, 과제 4는 버튼으로 PWM 듀티비를 바꾸며 C 디버거로 변수와 레지스터의 변화를 확인한다.[1]

폴링과 인터럽트

폴링은 메인 루프에서 입력과 시간을 반복해서 검사한다. 이번 설계는 PC13 입력의 상승 에지가 발생하면 버튼 ISR을 실행하고, TIM6 갱신 이벤트가 발생하면 타이머 ISR을 실행한다. polling_routine()에는 버튼 읽기와 LED 시간 판정 코드를 두지 않는다.[1]

인터럽트 우선순위와 공유 변수

프로젝트에서 TIM6의 우선순위는 4, 버튼은 7이다. 현재 NVIC 설정에서는 TIM6가 버튼 처리를 선점할 수 있다. 간격과 카운터는 두 ISR이 공유하므로 volatile로 선언하고, 두 값의 갱신은 짧은 임계 구역으로 묶는다. volatile만으로 여러 연산의 원자성이 보장되지는 않는다.[1] ISR은 짧게 실행한 뒤 복귀하도록 구성한다. 입력 대기, printf, HAL_Delay는 사용하지 않는다. 인터럽트 관리와 중단점의 학습 범위는 Cortex-M 교재의 7장과 14장에 해당한다.[4]

타이머와 PWM

타이머는 클록을 분주해 일정한 주기로 이벤트를 만든다. PWM은 주기를 유지하면서 HIGH 시간의 비율을 바꾼다. 듀티비 D는 HIGH 시간 tH를 전체 주기 T로 나눈 값이다. 이번 실험에서는 TIM6로 100 ms 기준을 만들고 TIM3 CH1으로 PB4에 1 kHz PWM을 출력한다.[1]

실험 장치와 개발 환경

항목	구성
보드	NUCLEO-G474RE
입력	PC13 USER 버튼 B1
과제 3 출력	PA5 내장 LED
과제 4 출력	PB4 TIM3_CH1 / D5
계측	오실로스코프 또는 로직 분석기
선택 부품	외부 LED, 330 Ω~1 kΩ 저항, 점퍼선
개발 도구	VS Code, STM32 HAL, CMake, Ninja, Arm GCC/GDB

2. 과제 3 인터럽트 LED

설계 목표와 시간 계산

처음에는 1초마다 LED 상태를 바꾼다. 버튼을 누를 때마다 1 → 2 → 4 → 8 → 1초로 순환하고, 입력이 없으면 선택한 간격을 유지한다. 켜짐과 꺼짐이 각각 한 간격이므로 전체 점멸 주기는 간격의 두 배이다.[1]

```
fTIM6 = 100,000,000 Hz
PSC = 9999, ARR = 999
T = (PSC+1)(ARR+1) / fTIM6
  = 10000 x 1000 / 100000000
  = 0.1 s = 100 ms
```

main.c의 클록과 TIM6 설정을 대입한 값이다. 100 ms 이벤트를 10, 20, 40, 80회 세면 각각 1, 2, 4, 8초가 된다.

회로 구성

과제 3은 보드의 PC13 사용자 버튼과 PA5 내장 LED를 사용한다. 별도의 외부 회로

없이 보드의 버튼과 LED로 구성한다.[1][3]

초기화 핵심 코드

```
/* ece2_474_init() 과제 3 부분 */
__HAL_TIM_SET_COUNTER(&htim6, 0U);
__HAL_TIM_CLEAR_FLAG(
    &htim6, TIM_FLAG_UPDATE);
if (HAL_TIM_Base_Start_IT(&htim6)
    != HAL_OK)
{
    Error_Handler();
}
```

초기 갱신 플래그를 지우고 인터럽트를 시작한다. 시작 전에 간격은 1000 ms, 카운터는 0, LED는 LOW로 초기화한다.

주기 인터럽트 핵심 코드

```
++tim6_irq_count;
++tim6_count;
if (tim6_count >=
    blink_interval_ms / 100U)
{
    tim6_count = 0U;
    HAL_GPIO_TogglePin(
        LED_ece2_GPIO_Port,
        LED_ece2_Pin);
    ++led_toggle_count;
}
```

위의 코드는 tim6_isr_p4()의 과제 3 처리하는 코드이다.

버튼 입력과 간격 변경

```
uint32_t now_ms = HAL_GetTick();
if (press_seen != 0U &&
    (uint32_t)(now_ms-last_press_ms)
        < 200U)
{
    return;
}
press_seen = 1U;
last_press_ms = now_ms;
++button_press_count;
```

첫 입력은 바로 허용한다. 이후 마지막 유효 입력으로부터 200 ms 이내의 반복 에지는 무시한다. PC13은 상승 에지로 검출하므로 계속 누르는 동안에는 새로운 유효 에지가 없으면 추가 변경하지 않는다. 접점 바운스 억제 효과는 실제 버튼으로 확인한다.

```
uint32_t saved_primask =
    __get_PRIMASK();
__disable_irq();
blink_interval_ms =
   (blink_interval_ms >= 8000U)
   ? 1000U : blink_interval_ms * 2U;
tim6_count = 0U;
__set_PRIMASK(saved_primask);
```

위의 코드는button_isr_p7()의 간격을 갱신하는 코드이다. 기존 인터럽트 마스크를 저장하고 복원하여 보호 구간 전의 상태를 유지한다.

선택 간격과 예상 출력

간격	TIM6 횟수	전체 주기
1 s	10	2 s
2 s	20	4 s
4 s	40	8 s
8 s	80	16 s

버튼이 눌리면 누적 횟수를 0으로 되돌린다. 다만 TIM6 자체의 진행 중인 100 ms 주기는 초기화하지 않는다. 따라서 변경 직후 첫 반전은 새 간격보다 최대 약 100 ms 먼저 발생할 수 있다. 이후 연속 반전 간격은 설정값을 기준으로 한다.

입력 시점부터 첫 반전까지의 시간과, 이후 정상 점멸의 전환 간격을 구분하여 기록한다. 버튼을 누를 때마다 LED를 즉시 반전하는 코드는 넣지 않았다.

과제 3 검증 항목

입력 또는 조건	예상 결과
리셋 후 시작	1000 ms 간격
버튼 없이 관찰	현재 간격으로 계속 점멸
버튼 1회씩 4회	2 → 4 → 8 → 1초

입력 또는 조건	예상 결과
버튼을 계속 누름	새 에지가 없으면 1회 변경
200 ms 이내 재검출	추가 변경 억제
각 간격 유지	TIM6 10/20/40/80회 마다 반전
간격 변경 직후	카운터 0, 새 간격 적용

시간 측정 방법

PA5의 연속 상승·하강 에지 사이를 측정해 상태 변경 간격을 구한다. 같은 방향의 에지 사이를 측정하면 전체 주기이므로 2로 나누어 간격과 비교한다. 각 단계에서 최소 3회 이상 측정하고 목표값, 측정값, 계측기 설정을 함께 기록한다.

간격 변경 직후 첫 전환은 TIM6의 기존 주기 위상에 영향을 받는다. 이 전환은 별도로 표시하고, 정상 간격의 오차는 이후 반복 구간에서 계산한다. 버튼 상태가 200 ms 이내에 여러 번 검출되는 경우는 입력 억제 조건의 확인에 사용한다.

```
상대 오차 [%]
= |측정 간격 - 목표 간격|
  / 목표 간격 x 100
```

3. 과제 4 PWM 비율 변경

설계 목표와 파라미터

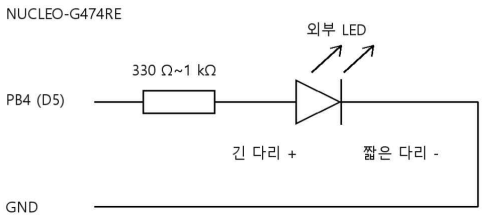
TIM3 CH1의 PWM을 PB4로 출력한다. 시작 듀티비는 25%이며, 사용자 버튼으로 50%, 75%, 25%를 순환한다. 과제 3과 4는 각각 다른 빌드 설정으로 실행하므로 버튼 하나가 두 과제를 동시에 수행하지 않는다.[1]

```
fTIM3 = 100,000,000 Hz
PSC = 99, ARR = 999
fPWM = fTIM3 / ((PSC+1)(ARR+1))
     = 1000 Hz
T = 1 ms
D = CCR1 / (ARR+1)
```

듀티비	CCR1	HIGH 시간
25%	250	0.25 ms
50%	500	0.50 ms
75%	750	0.75 ms

과제 4 회로 구성

과제 4는 계측기 입력을 PB4, 기준 접지를 GND에 연결한다. PB4는 PA5와 다른 출력 핀이므로 내장 LED로 PWM을 판정하지 않는다. Arduino 헤더에서는 D5에 해당한다.[1][3]



계측기 입력: PB4 기준 접지: GND

그림 1 과제 4의 선택적 외부 LED 연결과 측정점.

PB4 핀 대응은 보드 매뉴얼을 따른다.[3] 외부 LED는 밝기 변화를 보조적으로 관찰할 때 사용한다. 배선은 USB 전원을 분리한 상태에서 진행하며, LED의 긴 다리를 저항 쪽에 연결한다. 정확한 듀티비와 주파수는 파형으로 확인한다.

초기화 핵심 코드

```
/* 초기값 */
pwm_percent = 25U;
pwm_compare = 250U;
__HAL_TIM_SET_COMPARE(
    &htim3, TIM_CHANNEL_1,
    pwm_compare);
if (HAL_TIM_PWM_Start(
    &htim3, TIM_CHANNEL_1) != HAL_OK)
{
    Error_Handler();
}
```

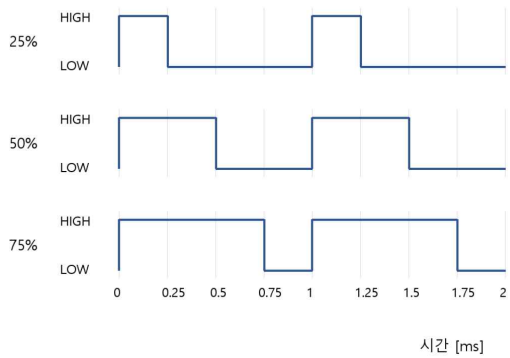
PWM 시작 전에 비교값을 설정해 초기 듀티비를 정한다. 출력의 반복은 하드웨어 타이머가 담당하므로 메인 루프에서 HIGH와 LOW를 번갈아 쓰지 않는다.

버튼 ISR 핵심 코드

```
pwm_percent = (pwm_percent >= 75U)
            ? 25U : pwm_percent+25U;
pwm_compare = pwm_percent * 10U;
__HAL_TIM_SET_COMPARE(
    &htim3, TIM_CHANNEL_1,
    pwm_compare);
```

과제 4는 공통 200 ms 입력 억제 처리 뒤 위 코드를 실행한다. CCR1의 값이 250 → 500 → 750 → 250으로 순환한다.

PWM 예상 파형



위의 그림은 1 kHz에서 듀티비별 이론적 예상 파형이다. 직접 측정한 파형이 아니며, 각 행은 동일 듀티비가 유지되는 두 주기를 나타낸다. 세 파형의 전체 주기는 모두 1 ms이다. 버튼 입력은 HIGH 시간의 비율만 바꾸므로 출력 주파수는 유지되어야 한다. 외부 LED를 연결하면 듀티비 증가에 따라 평균 전류와 밝기가 증가할 것으로 예상한다. 눈으로 느끼는 밝기가 듀티비에 정확히 비례하는 것은 아니다.

C 디버거 확인 절차

① 과제 4를 선택하고 F5로 시작한다. main()의 임시 중단점에서 멈추면 F5로 계속 실행한다.

② button_isr_p7()의 pwm_percent 갱신 줄에 F9로 중단점을 둔다.

③ USER 버튼을 눌러 ISR에서 멈추는지 확인한다.

④ Watch에 pwm_percent, pwm_compare, htim3.Instance->CCR1을 추가한다.

⑤ F10으로 한 줄씩 실행하며 변수와 비교 레지스터의 변화를 확인한다. 함수 내부 추적은 F11, 실행 재개는 F5이다.[1] 중단점에서는 해당 줄 실행 전의 값이 보일 수 있으므로 F10 이후의 값을 비교한다. 파형 측정 전에는 중단점을 해제하고 연속 실행한다. 디버거가 멈춘 구간을 정상 주기의 측정값으로 사용하지 않는다.[1]

과제 4 검증 항목

조건	예상 결과
리셋 후	25%, CCR1=250
첫 번째 누름	50%, CCR1=500
두 번째 누름	75%, CCR1=750
세 번째 누름	25%, CCR1=250
세 단계 공통	주파수 1 kHz, 주기 1 ms
버튼 없이 관찰	선택한 듀티비 유지
F10 한 줄 실행	변수와 CCR1 갱신 확인

파형 측정 방법

계측기 입력은 PB4에, 접지는 보드 GND에 연결한다. 연속 실행 상태에서 주기 T와 HIGH 시간 tH를 측정해 D = tH/T × 100%를 계산한다. 25%, 50%, 75%의 파형을 각각 저장하고 대응하는 CCR1 값을 함께 기록한다. 외부 LED 관찰만으로 정량 측정을 대신하지 않는다.

4. 소스 구성과 GITHUB저장소

파일	변경 또는 역할
ece2_474_routine.c	초기화, 버튼 ISR, TIM6 ISR, 관찰 변수 구현
CMakeLists.txt	STM32_1_TASK=3/4 선택
CMakePresets.json	Debug와 PWM 별도 빌드
ece2_474.code-workspace	Configure·Build·디버깅 작업 연결
.vscode/settings.json	사용할 CMake 경로 설정
main.c 및 HAL 파일	기존 타이머, GPIO 설정 사용

GITHUB 저장소와 태그 및 커밋 해시

- GITHUB주소
https://github.com/thechef030/STM32_1
- 제출 브랜치: stm32-1
- 전체 커밋 해시:
22cdb6b7dae93de8506f77084e07b34bbd36a5c8
-제출 태그: v1.1.0

5. 참고문헌

[1] 전전설II LAB5 교안

[2] STMicroelectronics, STM32CubeG4 MCU Firmware Package, GitHub. README의 CMSIS·HAL·LL 구성과 보드 예제 설명.

[3] STMicroelectronics, STM32G4 Nucleo-64 boards (MB1367), UM2505. NUCLEO-G474RE의 사용자 버튼, 내장 LED 및 PB4/D5 핀 연결.

[4] Joseph Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes, 2013. ISBN 9780124080829.

[5] 과제4의 회로 구성 및 PWM파형 그림 생성 Chat gpt 활용.